Лабораторная работа №06

Реализация освещения по методу Фонга.

Целью лабораторной работы №6 является реализация освещения по методу Фонга с использованием GLSL-шейдеров. Для выполнения лабораторной работы требуется реализовать ряд вспомогательных классов, в частности: класс для представления источника света (Light) и класс для работы с материалом (Material).

Также изменению подвергнется и ряд других классов: класс для представления графического объекта (GraphicObject), класс для работы с ресурсами (ResourceManager) и класс для взаимодействия с видеокартой (RenderManager).

Основные сведения о модели освещения Фонга.

В лабораторной работе №6 необходимо реализовать вывод объектов с использованием освещения по методу Фонга. Для реализации любого графического эффекта с использованием GLSL-шейдеров необходимо выполнить три основных действия:

1. Необходимо написать сам шейдер, который бы реализовывал нужный графический эффект. При этом шейдер, как и любая другая программа, является, по сути, алгоритмом, который выполняет ряд операций над входными данными по заранее определенным правилам для получения результирующего значения. Иными словами, при разработке шейдера необходимо определиться как с самим алгоритмом (используемыми формулами), так и с тем, какие данные необходимы шейдеру для вычисления результирующего значения.

   В одном из следующих разделов будет подробно объяснен алгоритм расчета освещения по Фонгу. Более того, в примере к данной лабораторной работе приводится пример шейдер, который рекомендуется использовать для тестирования работы программы на начальном этапе.

2. Необходимо выбрать шейдер в качестве текущего и установить все его параметры (все uniform-переменные). Для работы шейдера необходимо, чтобы основная программа передавала в шейдер все требуемые данные. При расчете освещения к таким данным относятся параметры материала и параметры источника света. Эти параметры являются общими для всего объекта, который будет выводится в дальнейшем.

   Особое внимание следует обратить на то, чтобы передаваемые данные передавались так, как на это рассчитывает шейдер. Например, если шейдер выполняет расчет освещения считая, что позиция источника света задана в системе координат наблюдателя, а не в глобальной системе координат, то основная программа должна передавать данные именно в этой системе координат. Это особенно важно при написании собственного шейдера путем заимствования кусков кода из существующих шейдеров. В этом случае необходимо иметь в виду назначение и формат всех данных, используемых шейдером.

3. Последним этапом вывода графического объекта на экран является вывод его меша. При выводе меша в вершинный шейдер передаются все атрибуты каждой вершины.

   Атрибуты вершины также могут принимать участие в расчете параметров выводимой модели. В частности, на освещенность объекта влияет такой важный параметры, как вектор нормали к поверхности. Соответственно, для того чтобы шейдер мог правильно рассчитать освещенность, для каждой вершины должен быть задан вектор нормали и, более того, он должен быть представлен в той системе координат, на которую рассчитывает шейдер.

Если все параметры (uniform и attribute переменные) переданы корректно, то в результате на экран будет выведен объект тем способом, который определен в шейдере. В последующих разделах будет рассмотрен как процесс передачи необходимых параметров из основной программы в шейдер, так и сам шейдер расчета освещения. В этом же разделе будут приведены общие сведения о модели освещения Фонга.

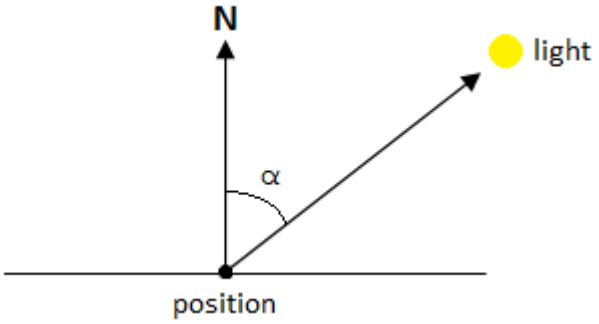
Для того, чтобы определить, какие именно данные должны быть переданы в шейдер, необходимо вспомнить формулу Фонга, а также то, от чего зависит освещенность объектов. Освещенность, рассчитанная по методу Фонга, зависит от трех основных составляющих, перечисленных ниже:

1. Материал – определяет отражающую способность объекта. На бытовом уровне под этим понимается цвет объекта. Разные объекты могут обладать разным цветом даже при одной и той же освещенности. Таким образом, каждый объект обладает своим собственным материалом.

2. Источник света – определяет, как освещены все объекты: это может быть тусклая желтая лампа, красное закатное солнце или направленный белый цвет прожектора. В каждом из этих случаев результирующее изображение при одних и тех же материалах будет существенно отличаться. Источник света, в отличие от материала, характеризуется не только своим цветом, но и положением.

3. Взаимное расположение источника света, объекта и наблюдателя – эти параметры являются хоть и менее очевидными, но не менее важными для правильного расчета освещенности, поскольку объект освещен сильнее если лучи от источника света падают на него под прямым углом. Также эти параметры оказывают влияние на определение блика на поверхности.

   Взаимное расположение, как правило, определяется с помощью различных векторов. Например, угол падения луча света на поверхность определяется как угол между направлением от указанной точки до источника света и нормалью к поверхности в этой точке.

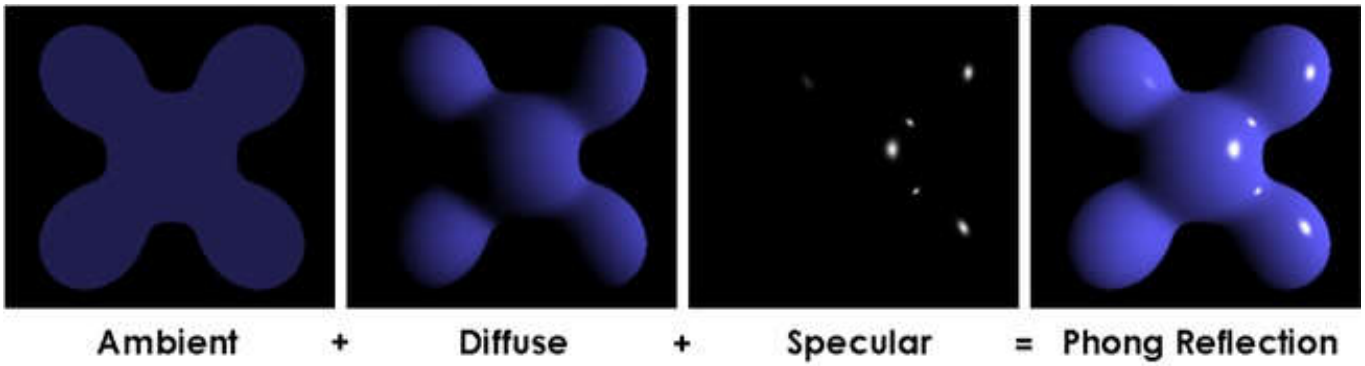


   Еще раз следует напомнить, что при выполнении любых операций над несколькими геометрическими данными, например, над вектором нормали, вектором направления на наблюдателя и вектором направления на источник света, необходимо, чтобы все они были выражены в общей системе координат.

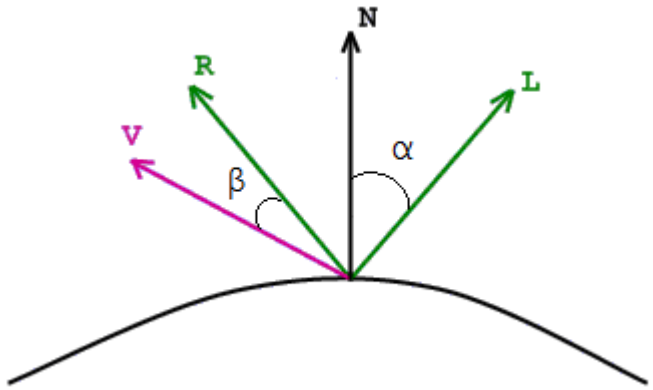
Модель освещения Фонга была предложена в попытке относительно простыми формулами добиться возможности представлять достаточно большой спектр материалов. Прежде всего, требуется иметь возможность задавать как матовые объекты, освещенность которых не зависит от взаимного расположения наблюдателя и источника света, так и глянцевые, которые отражают большинство лучей в определенном направлении, формируя блик на поверхности, если смотреть на них под определенным углом.

Для представления различных аспектов освещения (прежде всего, возможности задать матовые или зеркальные объекты) свет делят на три компонента: **фоновый**, **диффузный** и **зеркальный**.

Материал объекта также состоит из трех компонент, которые задают коэффициент отражения для фоновых, диффузных и зеркальных лучей источника света соответственно. Это позволяет, например, указать, что материал не отражает зеркальные лучи. В этом случае объект будет освещен одинаково, независимо от направления взгляда и будет казаться матовым. Или, наоборот, указать, что объект отражает почти все зеркальные лучи, что приведет к тому, что объект будет восприниматься глянцевым.



Для расчета освещенности (цвета) некоторой точки объекта необходимо не только знать параметры материала и цвет источника света, но и определить взаимное расположение этой точки, источника света и наблюдателя. В частности, освещенность любой точки вычисляется в соответствии с рисунком, представленным ниже:



L – направление от определенной точки к источнику света (направление, с которого падают лучи света);

R – отраженный относительно нормали вектор L (вектор идеального отражения, показывающий с какого направления будет наблюдаться наиболее сильный блик на поверхности);

V – направление на наблюдателя (вектор от заданной точки к наблюдателю);

N – вектор нормали к поверхности в точке, для которой рассчитывается освещенность. Вектор нормали задает ориентацию поверхности в этой точке.

Рассмотрим влияние каждой составляющей на итоговую освещенность объекта.

Диффузная (рассеянная) составляющая – это лучи, которые при падении на поверхность отражаются равномерно во все стороны. Освещенность объекта при этом зависит от угла падения луча света на поверхность, но не зависит от положения наблюдателя. Итоговая формула освещенности для лучей данного типа представлена ниже:

$$\mathbf{I_d = L_d * M_d * \cos(\alpha)}$$

α – угол падения луча света относительно нормали;

L_d – диффузная составляющая источника света;

M_d – диффузная отражающая способность материала;

Зеркальная составляющая – это лучи, которые ведут себя очень похоже на идеальные, то есть отражаются не во все стороны, а вдоль вектора идеального отражения, поэтому они влияют на воспринимаемую освещенность объекта только в том случае, если наблюдатель расположен в том же направлении. Данная составляющая, формирует блик на поверхности. Итоговая формула зеркальной составляющей представлена ниже:

$$\mathbf{I_s = L_s * M_s * \cos^p(\beta)}$$

в – угол между идеально отраженным лучом и вектором на наблюдателя;

L_s – зеркальная составляющая источника света;

M_s – зеркальная отражающая способность материала;

p – степень отполированности материала (величина блика).

Фоновая составляющая – это вторичные лучи, то есть лучи, которые падают на объект со всех сторон, после того как были отражены другими объектами. Фоновая составляющая необходима для того, чтобы поверхности, на которые не падают прямые лучи от источника света, все равно не были абсолютно черными, а были хоть чуть-чуть, но освещены. Интенсивность таких лучей, как правило, невелика. Формула расчета выглядит следующим образом:

$$\mathbf{I_a = L_a * M_a}$$

La – фоновая составляющая источника света;

Ma – фоновая отражающая способность материала;

Итоговая формула освещенности, которая и будет реализована в данной лабораторной работе, выглядит следующим образом:

$$\mathbf{I = L_d * M_d * \cos(\alpha) + L_s * M_s * \cos^p(\beta) + L_a * M_a}$$

L_d, L_s, L_a – параметры источника света;

M_d, M_s, M_a и p – параметры материала;

α и в – углы, определяемые взаимным расположение источника света, объекта и наблюдателя

Преобразование геометрических данных.

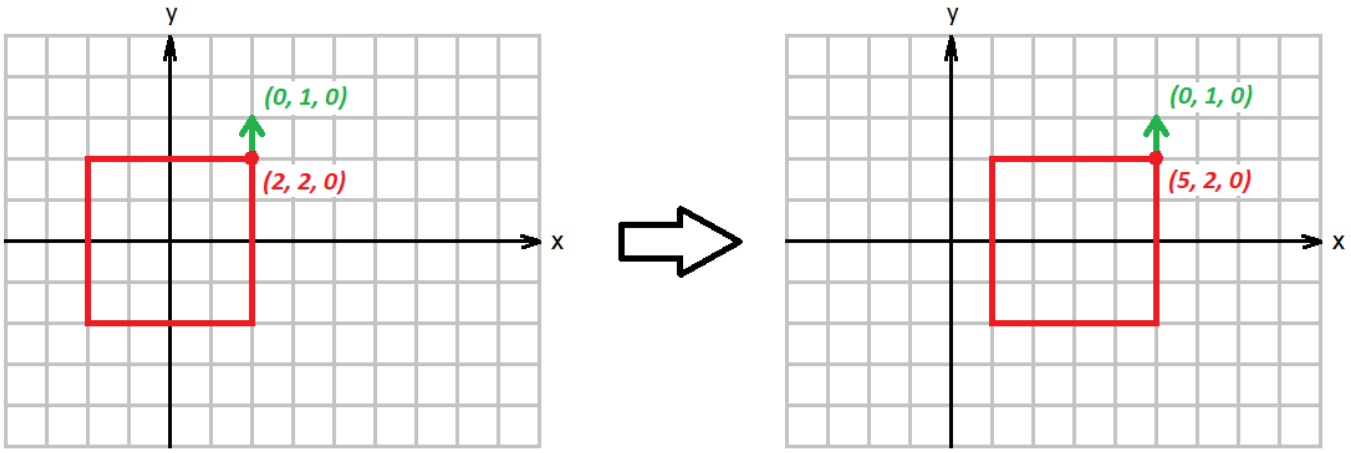
Одной из основных задач при вычислении освещения по методу Фонга является вычисление углов между различными векторами. Например, для вычисления диффузной составляющей освещения необходимо вычислить косинус угла между вектором нормали и направлением на источник света в точке, для которой рассчитывается освещенность. Совершенно очевидно, что для корректного расчета необходимо, чтобы все геометрические данные были преобразованы в общую систему координат. Вместе с тем, в приложениях трехмерной компьютерной графики традиционно применяются несколько различных систем координат:

1. Система координат модели (model space). Каждая модель (меш) проектируется в своей собственной системе координат, поэтому такие параметры, как позиция вершины или вектор нормали, задаются именно в этой системе координат.
2. Глобальная система координат (world space). Для размещения объектов на сцене используется глобальная система координат, которая позволяет расположить различные объекты друг относительно друга, то есть для вывода объектов на экран они должны быть преобразованы из системы координат модели в глобальную систему координат. Кроме того, в глобальной системе координат удобно задавать общие для всей сцены параметры, такие как, например, источник света или позицию наблюдателя.
3. Система координат наблюдателя (view space). В конечном счете, графические объекты выводятся на экран именно с позиции наблюдателя, поэтому все вершины графических объектов рано или поздно должны быть преобразованы в эту систему координат.
4. В завершении следует также упомянуть и другие системы координат, используемые OpenGL для проецирования трехмерных геометрических координат на двухмерную плоскость экрана. В частности, к этим системам координат относятся: усеченная система координат (clip space), система нормализованных координат устройства (ndc, normalized device coordinates) и система координат порта просмотра (viewport space). Однако, эти системы координат не подходят для расчета освещения, поскольку они искажают пропорции объектов и изменяют углы между векторами, за счет чего достигается эффект перспективы.

Иными словами, для выполнения операция над геометрическими данными необходимо выбрать одну из первых трех упомянутых систем координат. Как правило, для этого используется система координат наблюдателя по целому ряду причин:

1. Все вершины в любом случае должны быть преобразованы в систему координат наблюдателя, поэтому и расчет освещения можно производить в этой системе координат;
2. Позиция наблюдателя в системе координат наблюдателя всегда постоянна и совпадает с началом этой системы координат (то есть с точкой (0.0, 0.0, 0.0)), поэтому данный параметр вообще можно не передавать в шейдер;
3. Позицию источника света (или направление на источник света) можно достаточно легко перевести в систему координат наблюдателя, просто умножив её на матрицу наблюдения (матрицу камеры).

Геометрические данные, с которыми работают программы трехмерной графики, можно разделить на два типа – это точки в пространстве и направления. Например, позиция вершины меша – это геометрическая точка, а вектор нормали – это направление. И те и другие данные задаются с помощью векторов, но ведут себя по-разному. Например, при перемещении объекта вбок без вращения позиция вершины меняется, а значение вектор нормали остается неизменным:



Вместе с тем, при повороте меняется как позиция вершины, так и направление вектора нормали. Иными словами, при трансформации вершины необходимо учитывать как поворот, так и смещение, в то время как для направлений необходимо учитывать только поворот.

Наличие в компьютерной график как точек, так и направлений – это одна из причин использования однородных координат для указания геометрических данных. В этом случае для точек последнее значение в векторе (x, y, z, w) равно единице, а для направлений – нулю.

Для рисунка выше позиция вершины может быть закодирована вектором (2, 2, 0, 1), а вектор нормали – вектором (0, 1, 0, 0). Это позволяет обрабатывать и точки, и направления с использованием одних и тех же матриц преобразования. Ниже приведен пример умножения обоих векторов на матрицу трансформации, задающее указанное перемещение:

$$\begin{bmatrix} 1 & 0 & 0 & 3 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 5 \\ 2 \\ 0 \\ 1 \end{bmatrix}$$

преобразование позиции вершины

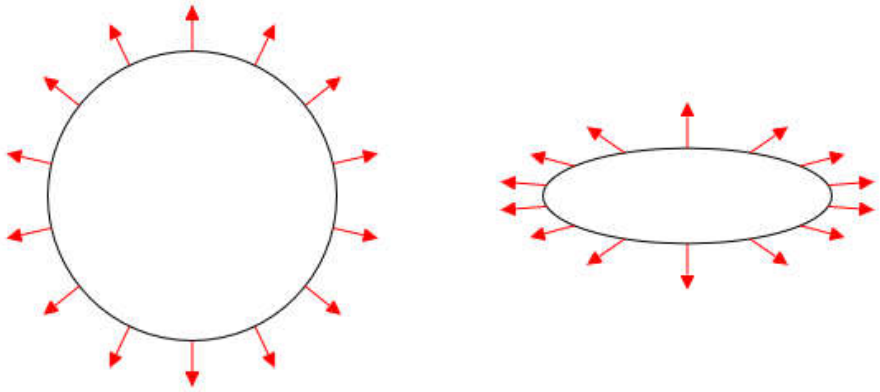
$$\begin{bmatrix} 1 & 0 & 0 & 3 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

преобразование вектора нормали

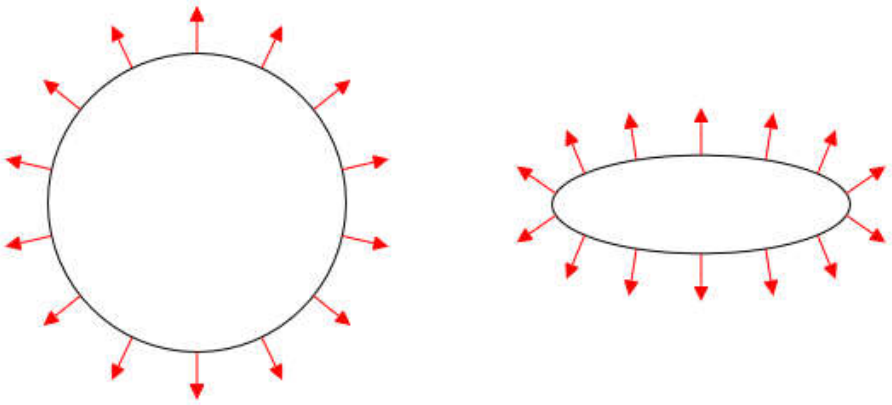
Позиция точек меняется как при перемещении системы координат, так и при её вращении, в то время как на направления смещение системы координат не оказывает влияния, поскольку последний столбец матрицы, который как раз и задает перемещение, всегда умножается на ноль.

Приведенный выше пример, хоть и иллюстрирует разницу между точками и направлениями, является не совсем корректным. В частности, в матрице трансформации, помимо поворота и переноса, также может быть закодировано масштабирование или искажение осей. В этом случае, если просто умножить нормаль на матрицу трансформации, нормаль окажется не перпендикулярной к поверхности.

На рисунке снизу показан пример сжатия исходной модели. В первом случае нормаль подвергается той же самой трансформации, что и сам объект, в результате чего нормали больше не перпендикулярны поверхности, что противоречит их сути:



Корректное преобразование нормалей должно происходить таким образом, чтобы они оставались перпендикулярными к поверхности:



Иными словами, для преобразования нормалей должна использоваться несколько иная матрица, которая называется матрицей нормалей (normal matrix) и которая может быть получена из матрицы наблюдения модели последовательным получением обратной матрицы и затем её транспонированием: $(M^{-1})^T$, где M – исходная матрица наблюдения модели, применяемая для преобразования вершин.

Здесь не приводится математическое обоснование указанного способа преобразования нормали. Дополнительную информацию можно получить, например, по следующей ссылке:

https://paroj.github.io/gltut/Illumination/Tut09%20Normal%20Transformation.html

Следует, однако, отметить, что вышеприведенный способ получения матрицы нормалей необходим только в том случае, если объект действительно подвергается искажению или масштабированию. В противном случае можно использовать обычную матрицу наблюдения модели, что является, безусловно, более быстрым и простым способом. В большинстве случаев используется именно этот способ.

Передача параметров материала.

В соответствии с моделью освещения Фонга для придания большей вариативности освещенность делится на несколько компонент: фоновая (ambient), диффузная (diffuse) и зеркальная (specular) компонента. Каждая компонента по-прежнему задается вектором (r, g, b, a), то есть можно говорить о зеленых фоновых лучах, красных диффузных лучах и фиолетовых зеркальных лучах, которые совместно определяют освещенность какого-либо объекта. Иногда к модели освещенности также добавляют цвет самосвечения объекта (emission), который в данной лабораторной работе не используется.

Считается, что источник света испускает лучи каждого типа с разной интенсивностью, а материал объекта способен отразить эти лучи в разной пропорции. Таким образом, задается отражающая способность для каждой из трех компонент освещенности: отдельно для фоновой, отдельно для диффузной и отдельно для зеркальной составляющей освещенности. Кроме того, для материала также указывается степень отполированности объекта (shininess), которая определяет концентрированность блика на поверхности.

Для организации хранения параметров материала необходимо использовать отдельный класс, представленный ниже:

```
// КЛАСС ДЛЯ РАБОТЫ С МАТЕРИАЛОМ
class Material
{
public:
	// конструктор по умолчанию
	Material ();

	// загрузка параметров материала из json-файла
	bool loadFromJson(std::string filename);

	// получение различных параметров материала
	glm::vec4& getAmbient();
	glm::vec4& getDiffuse();
	glm::vec4& getSpecular();
	float getShininess();

private:
	// фоновая составляющая
	glm::vec4 ambient;
	// диффузная составляющая
	glm::vec4 diffuse;
	// зеркальная составляющая
	glm::vec4 specular;
	// степень отполированности
	GLfloat shininess;

};
```

При работе с материалом в данной лабораторной работе необходимо обратить внимание на два основных момента:

1. Загрузка параметров материала производится из json-файла, за что отвечает метод loadFromJson. Для парсинга json-файла рекомендуется использовать библиотеку RapidJson, пример работы с которой был рассмотрен в проекте прошлого семестра.

2. Материал является еще одним ресурсом программы, поэтому загрузка материалов должна осуществляться с использование класса ResourceManager также, как это происходит с мешами и текстурами, то есть менеджер ресурсов должен следить за тем, какие материалы уже загружены и на загружать материалы повторно. При этом при загрузке материала менеджер ресурсов возвращает его идентификатор в своем внутреннем хранилище.

В завершении следует отметить, что материал является частью графического объекта, то есть каждый графический объект определяется мешем, материалом, текстурой, своей позицией и ориентацией в пространстве, а также еще некоторыми параметрами, часть из которых будет добавлена в следующей лабораторной работе. На данном этапе необходимо добавить в класс графического объекта необходимые поля и методы, в результате чего класс будет выглядеть следующим образом:

```
// КЛАСС ДЛЯ РАБОТЫ С ГРАФИЧЕСКИМ ОБЪЕКТОМ
class GraphicObject
{
public:
	// конструктор по умолчанию
	GraphicObject();

	// установить цвет объекта
	void setColor(glm::vec4 &color);
	// установить позицию объекта
	void setPosition(glm::vec3 &position);
	// установить угол поворота в градусах относительно оси OY по часовой стрелке
	void setAngle(float degree);

	// установить идентификаторы
	void setMeshId(int id);
	void setTextureId(int id);
	void setMaterialId(int id);

	// получить параметры и идентификаторы ресурсов
	glm::vec4& getColor();
	glm::mat4& getModelMatrix();
	int getMeshId();
	int getTextureId();
	int getMaterialId();

private:
	// идентификатор используемого меша
	int meshId;
	// идентификатор используемой текстуры
	int textureId;
	// идентификатор используемого материала
	int materialId;
	// цвет объекта
	glm::vec4 color;
	// матрица модели (задает позицию и ориентацию)
	glm::mat4 modelMatrix;

};
```

Важно отметить, что такие поля графического объекта как текстура, материал и меш представлены не самими сущностями, которые являются достаточно сложными, а их идентификаторами в менеджере ресурсов. Это позволяет, с одной стороны, не дублировать одинаковые сущности для нескольких графических объектов, а, с другой стороны, позволяет легко определить, что два графических объекта используют, например, одинаковый материал. Последняя возможность будет использоваться в следующей лабораторной работе для оптимизации вывода сложной сцены.

Передача параметров источника света.

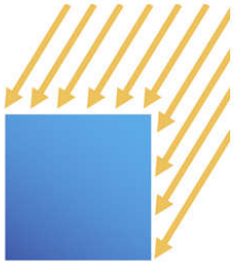
Источник света, также как и материал, обладает фоновой, зеркальной и диффузной компонентой. Кроме того, он имеет еще один обязательный параметр, который определяет, где находится источник света.

В компьютерной графике принято различать два типа источников света: точечные источники света (positional light) и направленные источники света (directional light).

Positional Light

Точечные источники света определяются своей позицией и могут служить для имитации, например, лампочки под потолком. Такой источник света находится в некоторой геометрической точке, позиция которой задается вектором (x, y, z, 1), то есть последняя компонента в векторе однородных координат равна единице.

Направленные источники света используются для имитации источников света, находящихся настолько далеко от освещаемой поверхности, что все лучи света, падающие на объект, можно считать параллельными. В этом случае задается не позиция источника света, а направление на него в виде вектора (x, y, z, 0). Например, для задания источника света, представленного солнцем, находящемся в зените, можно использовать вектор (0, 1, 0, 0).



Directional Light

Как бы то ни было, и позиция, и направление на источник света, обычно задаются в глобальной системе координат, поэтому, как было указанно ранее, при передаче этого параметра в шейдер его необходимо умножить на матрицу камеры, чтобы перевести в систему координат наблюдателя.

В данной лабораторной, для работы с направленным источником света, необходимо реализовать класс, структура которого выглядит следующим образом:

```
// КЛАСС ДЛЯ РАБОТЫ С ИСТОЧНИКОМ СВЕТА
class Light
{
public:
        // конструктор по умолчанию
        Light ();

        // задание различных параметров источника света
        void setDirection(glm::vec3& direction);
        void setAmbient(glm::vec4& ambient);
        void setDiffuse(glm::vec4& diffuse);
        void setSpecular(glm::vec4& specular);

        // получение различных параметров источника света
        glm::vec4& getDirection();
        glm::vec4& getAmbient();
        glm::vec4& getDiffuse();
        glm::vec4& getSpecular();

private:
        // позиция источника света
        glm::vec4 direction;
        // фоновая составляющая источника света
        glm::vec4 ambient;
        // диффузная составляющая
        glm::vec4 diffuse;
        // зеркальная составляющая
        glm::vec4 specular;

};
```

Шейдер расчета освещения по Фонгу.

Прежде чем приступить к написанию шейдера, необходимо отметить, что существует два способа вычисление освещения объекта. В первом случае, как это происходит в фиксированном конвейере рендеринга OpenGL, освещенность по формуле Фонга рассчитывается для каждой вершины, после чего происходит интерполяция цвета на весь полигон. Данный способ, как правило, показывает выигрыш в производительности, но приводит к появлению артефактов изображения, заключающихся в визуально заметной смене освещенности на границе полигонов. Для того, чтобы избежать этих недостатков, освещенность, чаще всего, рассчитывают во фрагментном шейдере, отдельно для каждого фрагмента.

Вершинный шейдер.

Результат вывода любого объекта на экран определяется тем, какой шейдер выбран в качестве текущего перед отрисовкой полигонов. При этом процесс вывода полигонов объекта начинается с вершинного шейдера, который обрабатывает каждую вершину. При этом, вершинный шейдер должен решить две основные задачи:

1. Вычислить значение встроенной переменной gl_Position, которая косвенно определяет позицию вершины на экране. Вычисление данной переменной является обязательной частью и в данной лабораторной работе выполняется также, как и ранее, то есть с использованием матрицы проекции, матрицы наблюдения модели и геометрических координат каждой вершины.
2. Определить значение всех переменных, которые необходимо передать из вершинного шейдера во фрагментный. Для расчета освещения фрагментному шейдеру, помимо параметров источника света и материала, необходимо знать позицию и вектор нормали для каждого фрагмента. Эти значения рассчитываются для каждой вершины и передаются из вершинного шейдера во фрагментный. Для передачи указанных параметров в вершинном шейдере объявлены две переменные: position и normal. Кроме того, также как и в предыдущей лабораторной работе, шейдер накладывает текстуру, поэтому вершинный шейдер вычисляет и передает во фрагментный шейдер текстурные координаты через переменную texCoord.

Полный текст вершинного шейдера выглядит следующим образом:

```
#version 330 core

uniform mat4 projectionMatrix;
uniform mat4 modelViewMatrix;

layout(location = 0) in vec3 vPosition;
layout(location = 1) in vec3 vNormal;
layout(location = 2) in vec2 vTexCoord;

out vec3 position;
out vec3 normal;
out vec2 texCoord;

void main ()
{
    position = vec3(modelViewMatrix * vec4 (vPosition, 1));
    normal = vec3(modelViewMatrix * vec4 (vNormal, 0));
    texCoord = vec2(vTexCoord.s, 1.0 - vTexCoord.t);

    gl_Position = projectionMatrix * modelViewMatrix * vec4 (vPosition, 1) ;
}
```

К вершинному шейдеру необходимо сделать несколько замечаний.

- Во-первых, для выполнения всех обязанностей, возложенных на вершинный шейдер, ему необходимы только матрицы преобразования и атрибуты вершины, поэтому в данном шейдере не объявлены параметры материала и параметры источника света. Эти параметры будут объявлены непосредственно во фрагментном шейдере.

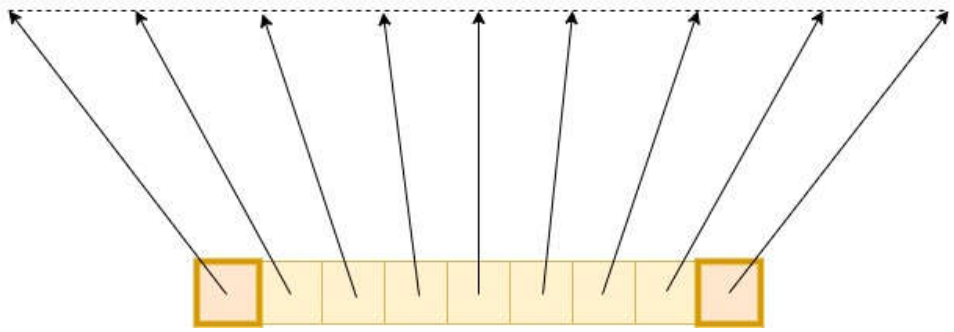
- И позиция, и нормаль умножаются на одну и туже матрицу наблюдения модели, которая переводит их из системы координат модели в систему координат наблюдателя. Следует отметить, что при вычислении позиции вершины последний компонент вектора равен единице, потому что позиция вершины является геометрической точкой в пространстве и при её преобразовании необходимо учитывать как поворот, так и перенос системы координат. В то же время, при вычислении вектора нормали последний компонент вектора равен нулю, потому что вектор нормали является направлением, и при его преобразовании необходимо учитывать только поворот системы координат, но не её перемещение.

- Для передачи позиции вершины и вектора нормали из вершинного шейдера во фрагментный они должны быть объявлены с модификатором out в вершинном шейдере и с модификатором in во фрагментном шейдере. Тем не менее, значение из вершинного шейдера во фрагментный передаются не напрямую, а после растеризации и линейного интерполирования. Для лучшего понимания механизма передачи параметров можно более подробно рассмотреть это процесс:

  1. Для всех вершин, составляющих полигон, вершинный шейдер вычисляет все переменные, объявленные в нем с модификатором out. В данном случае такими параметрами будут вектор нормали и позиция вершины.

  2. После обработки всех вершин, OpenGL разбивает полигон на множество фрагментов, каждый из которых соответствует определенному пикселю экрана. Этот процесс называется растеризацией.

  3. У каждого фрагмента есть свой собственный экземпляр всех ранее рассчитанных параметров, однако данное значение не равно одному из значений, полученных для вершин, а рассчитано таким образом, чтобы оно плавно менялось от одной вершины к другой. Данный способ называется линейным интерполированием и является логичным решением для вычисления соответствующих значений каждого фрагмента.



  Следует отметить, что даже если нормали имеют единичную длину, но направленны под углом к полигону (что может использоваться для моделирования изогнутых поверхностей), то нормали для фрагментов буду иметь меньшую длину, и в этом случае во фрагментном шейдере их необходимо дополнительно нормализовать для получения векторов с единичной длиной.

- В завершении стоит также отметить, что вершинный шейдер по-прежнему вычисляет и передает во фрагментный шейдер текстурные координат, чтобы фрагментный шейдер мог использовать текстуру при вычислении цвета фрагментов. Текстурные координаты преобразуются и передаются также, как это происходило в предыдущей лабораторной работе.

Фрагментный шейдер.

Фрагментный шейдер занимается тем, что на основании переданных ему uniform-переменных и переменных, полученных от вершинного шейдера, вычисляет цвет фрагмента. В частности, для вычисления цвета в соответствии с формулой Фонга вершинному шейдеру необходимы параметры материала, параметры источника света, позиция фрагмента и вектор нормали. Кроме того, в данной лабораторной также реализовывается наложение текстуры. В результате полный текст фрагментного шейдера выглядит следующим образом:

```
1  #version 330 core
2
3  uniform sampler2D texture_0;
4
5  uniform vec4 mAmbient;
6  uniform vec4 mDiffuse;
7  uniform vec4 mSpecular;
8  uniform float mShininess;
9
10 uniform vec4 lAmbient;
11 uniform vec4 lDiffuse;
12 uniform vec4 lSpecular;
13 uniform vec4 lPosition;
14
15 in vec3 position;
16 in vec3 normal;
17 in vec2 texCoord;
18
19 out vec4 outputColor;
20
21 void main (void)
22 {
23     vec3 n_Normal = normalize(normal);
24     vec3 n_ToEye = normalize (vec3(0,0,0) - position);
25     vec3 n_ToLight = normalize(vec3(lPosition));
26     vec3 n_Reflect = normalize(reflect(-n_ToLight,n_Normal));
27
28     vec4 ambient = mAmbient * lAmbient;
29     vec4 diffuse = mDiffuse * lDiffuse * max(dot(n_Normal, n_ToLight), 0.0f);
30     vec4 specular = mSpecular * lSpecular * pow(max(dot(n_ToEye, n_Reflect), 0.0f), mShininess);
31
32     vec3 color = vec3(ambient + diffuse + specular);
33     float alpha = mDiffuse.a;
34
35     vec4 texColor = texture(texture_0, texCoord);
36     color = color * vec3(texColor);
37
38     outputColor = vec4(color, alpha);
39 }
```

К фрагментному шейдеру необходимо также сделать несколько замечаний:

- Прежде всего необходимо обратить внимание на строки 3 – 13, где происходит объявление всех uniform-переменных. Для расчета освещения и наложения текстуры фрагментному шейдеру требуются все параметры материала (4 штуки), все параметры источника света (4 штуки), а также переменная типа sampler2D для обращения к текстуре.
- Кроме того, в строках 15 – 17 объявлены переменные, полученные от вершинного шейдера. К таким переменным относятся: позиция фрагмента в глобальной системе координат, нормаль к полигону в данной точке и вектор текстурных координат именно для обрабатываемого фрагмента.
- В основной функции фрагментного шейдера, то есть в функции main, для начала необходимо определить все вектора, участвующие в вычислении освещенности, что происходит в строках 23 – 26. Вектора используются для определения косинуса различных углов, поэтому необходимо получить единичные вектора, что дает возможность использовать скалярное произведение двух единичных векторов для вычисления косинус угла между ними.

Для расчета освещения необходимы следующие вектора:

1. Вектор нормали (n_Normal). Вектор нормали для каждого фрагмента передается из вершинного шейдера, но, как было показано ранее, в процессе интерполяции может изменить свою длину, поэтому его необходимо заново нормализовать, что выполняется с помощью встроенной функции GLSL, которая называется normalize:

   vec3 n_Normal = normalize(normal);

2. Направление на наблюдателя (n_ToEye). Направление на наблюдателя может быть получено как вектор из текущего фрагмента (его позиция передается из вершинного шейдера в переменной position) к наблюдателю. С учетом того, что все вычисления выполняются в системе координат наблюдателя, его позиция совпадает с центром системы координат, то есть равна (0, 0, 0). Данный вектор также необходимо нормализовать:

   vec3 n_ToEye = normalize (vec3(0,0,0) - position);

3. Вектор направления на источник света (n_ToLight). Этот параметр передается непосредственно из основной программы. С учетом того, что используется направленный источник света, для каждого фрагмента направление на источник света будет равно переданному значению. На всякий случай его тоже можно нормализовать:

   vec3 n_ToLight = normalize(vec3(lPosition));

4. Отраженный вектор на источник света (n_Reflect). Для нахождения отраженного луча света можно воспользоваться еще одной встроенной функцией GLSL – reflect. Данная функция принимает два параметра: падающий вектор и вектор нормали, относительно которого необходимо отразить падающий вектор. Поскольку вектор должен быть именно падающим, то направление на источник света необходимо взять со знаком минус. Полученное значение, опять-таки, на всякий случай нормализуется:

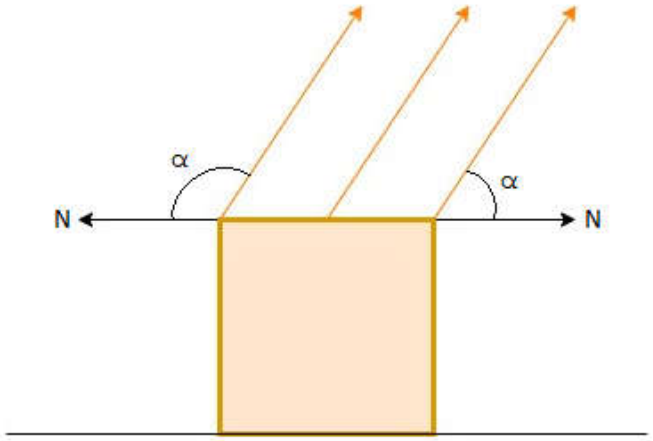
   vec3 n_Reflect = normalize(reflect(-n_ToLight,n_Normal));

- Далее происходит вычисление различных компонент освещенности в соответствии с формулой Фонга (строки 28 – 30). Для начала рассмотрим вычисление фоновой составляющей, поскольку она является самой простой. Фоновые лучи источника света падают на объект, который отражает определенный процент этих лучей в соответствии с фоновой отражающей способностью материала, то есть для вычисления фоновой составляющей достаточно перемножить соответствующие параметры источника света и материала:

  *vec4 ambient = mAmbient * lAmbient;*

- Диффузная составляющая вычисляется аналогичным образом, но здесь также необходимо учесть косинус угла между падающим лучом и вектором нормали (cos(α)). Если оба вектора имеют единичную длину, то косинус этого угла можно получить с помощью скалярного произведения векторов, которое в GLSL реализуется с помощью встроенной функции dot. Тем не менее, здесь кроется небольшая тонкость, связанная с тем, что для некоторых поверхностей лучи падают под углами, большими 90 градусов, что приводит к отрицательному косинусу.

  На рисунке, приведенном на следующей странице, показано, что при расчете освещения для правой грани угол α меньше девяноста градусов и косинус будет положительным, в то время как для левой грани угол больше девяносто градусов и косинус будет отрицательным.



Иными словами, необходимо сделать так, чтобы при отрицательном косинусе диффузная освещенность в этой точке была нулевой, поскольку это говорит о том, что на поверхность лучи света не падают. Сделать это можно, например, с использованием условного оператора if, который присутствует в GLSL. Тем не менее, шейдеры не очень производительно обрабатывают эту конструкцию, поэтому рекомендуется использовать функцию max, которая возвращает наибольшее из двух значений. В результате участок кода по вычислению диффузной составляющей выглядит следующим образом:

*vec4 diffuse = mDiffuse * lDiffuse * max(dot(n_Normal, n_ToLight), 0.0f);*

- Зеркальная составляющая рассчитывается аналогичным образом. При этом для возведения какого-либо значения в определенную степень используется встроенная функция pow. В результате расчет зеркальной составляющей выглядит следующим образом:

  *vec4 specular = mSpecular * lSpecular * pow(max(dot(n_ToEye, n_Reflect), 0.0f), mShininess);*

- Итоговая освещенность определяется суммой всех составляющих освещенности по Фонгу (строки 32 – 33). Кроме того, необходимо определить степень прозрачности фрагмента, даже если это пока не влияет на итоговый результат. Традиционно степень прозрачности задается как альфа-компонента диффузной составляющей материала.

- В строке 35, также как и ранее, происходит получения значения из текстуры. Значение, полученное из текстуры, необходимо каким-либо образом скомбинировать с рассчитанной освещенностью по методу Фонга. Для этого, как правило, оба значения перемножаются, чтобы одновременно учесть и освещенность фрагмента, и его индивидуальный цвет, заданный в текстуре (строка 36):

  vec4 texColor = texture(texture_0, texCoord);
  *color = color * vec3(texColor);*

- В завершении необходимо заполнить единственную выходную переменную фрагментного шейдера, которая и будет считаться цветом фрагмента (строке 38).

В завершении стоит отметить, что шейдер, приведенный в этом разделе, является учебным примером и поэтому не является оптимальным, с точки зрения производительности, однако хорошо демонстрирует последовательность вычислений для определения освещенности по методу Фонга.

Создание эффекта воздушной перспективы.

При выводе на экран достаточно больших сцен, простирающихся до самого горизонта (что будет реализовано в следующей лабораторной работе) можно заметить неприятный артефакт изображения, который связан с тем, что объекты, расположенные у дальней плоскости отсечения, будут отсечены OpenGL. При этом возможна ситуация, когда отсекается только часть объекта, если он пересекает дальнюю плоскость отсечения:

Данную проблему можно было бы попытаться решить, отодвинув дальнюю плоскость отсечения еще дальше, но это приводит к целому ряду негативных последствий. Во-первых, увеличивается количество объектов, попадающих в усеченную пирамиду видимости, что негативно сказывается на производительности. Во-вторых, уменьшается точность представления глубины объектов, что сказывается на качестве осуществления теста глубины. В результате, при выводе удаленных объектов их глубина вычисляется с большой погрешностью, что приводит к неприятным мерцаниям, когда то одни объекты, то другие, проходят тест глубины и сменяют друг друга в буфере кадра.

Одним из вариантов решения данной проблемы является использование эффекта воздушной перспективы (также называемым эффектом тумана). Воздушная перспектива характеризуется исчезновением четкости и ясности очертаний предметов по мере их удаления от глаз наблюдателя. При этом дальний план характеризуется уменьшением насыщенности цвета, яркости и четкости объектов. Эффект воздушной перспективы имеет и физическое обоснование, связанное с рассеиванием лучей света в неоднородной среде, содержащей примеси газов или пыли.

Воздушная перспектива является достаточно простым эффектом и для его реализации достаточно смешать во фрагментном шейдере вычисленный цвет фрагмента с цветом тумана в определенной пропорции. Смешивание цветов в этом случае выполняется по следующей формуле:

Color = FragmentColor * (1.0 – A) + FogColor * A

Коэффициент A – это коэффициент затуманивания, который определяется расстоянием от наблюдателя до вычисляемого фрагмента и меняется от нуля до единицы. Фрагменты, которые лежат ближе к наблюдателю, имеют меньший коэффициент затуманивания и видны достаточно четко, в то время как фрагменты, лежащие на дальней плоскости отсечения, имеют коэффициент затуманивания близкий к единице и почти сливаются с фоном.

Непосредственно для смешивания цветов вместо того, чтобы самостоятельно вычислять указанную формулу, что требует выполнения четырех математических операций, можно воспользоваться встроенной в GLSL функцией mix, для которой есть шанс, что она выполняется за одно действие. Детали функции можно получить по следующей ссылке:

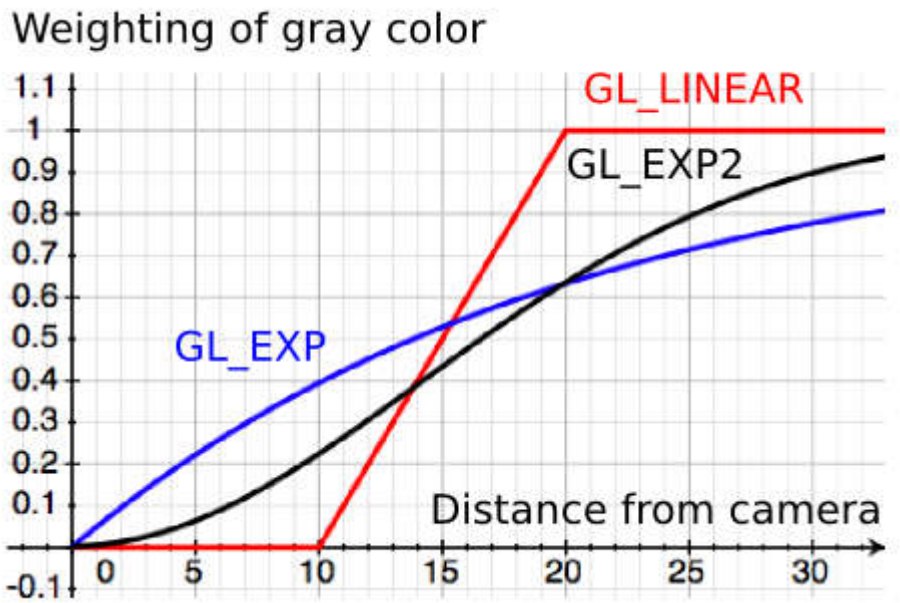
https://www.khronos.org/registry/OpenGL-Refpages/gl4/html/mix.xhtml

Эффект воздушной перспективы обычно характеризуется следующими параметрами, которые должны быть переданы в шейдер и учтены при расчете результирующего цвета фрагмента:

- Минимальная дистанция – дистанция, при которой начинает действовать эффект затуманивания. Фрагменты с дистанцией до наблюдателя меньшей, чем указанное значение, не подвергаются затуманиванию, для них коэффициент затуманивания равен нулю.
- Максимальная дистанция – дистанция, после которой фрагмент полностью теряет свой цвет и выводится с цветом тумана, то есть для таких фрагментов коэффициент затуманивания равен единице. Максимальная дистанция, как правило, подбирается так, чтобы объекты, лежащие рядом с дальней плоскостью отсечения, были полностью затуманены.
- Цвет тумана - цвет тумана может быть обусловлен средой, в которой находится объект. Для ясного дня, как правило, цвет тумана устанавливают в бледно голубой, что связанно с тем, что синий цвет меньше подвержен рассеиванию в земной атмосфере. Другими примерами может быть желтый цвет для песчаной бури, или зеленый цвет для имитации кислотного тумана.

- Плотность тумана – значение, определяющее интенсивность затуманивания при изменении дистанции от минимальной до максимальной. Разброс значения определяется формулой, используемой для расчета коэффициента затуманивания.

Отдельного внимания заслуживает вычисление коэффициента затуманивания, который меняется в зависимости от дистанции между вычисляемым фрагментом и наблюдателем. Дистанция и коэффициент затуманивания могут быть связанны как линейно, так и по более сложным формулам, что позволит сделать так, чтобы затуманивание происходило более интенсивно по мере удаления от наблюдателя:



Правильно подобранные параметры эффекта воздушной перспективы не только скрывают неточности отсечения удаленных объектов, но и позволяют лучше прочувствовать глубину сцены. В некоторых случаях эффект воздушной перспективы может иметь самостоятельную художественную ценность. Снизу приведено изображение из весьма атмосферной игры The Final Station:

Задание к лабораторной работе.

Лабораторная работа №6 строится на основе предыдущей работы. К программе предъявляются следующие требования:

1. Необходимо реализовать класс Light для работы с источником света. Класс должен иметь структуру, приведенную в соответствующем разделе. В лабораторной работе необходимо реализовать направленный источник света.
2. Необходимо реализовать класс Material для работы с материалом. Класс должен иметь структуру, приведенную в соответствующем разделе. Особое внимание необходимо обратить на то, что параметры материала должны загружаться из json-файла.
3. Необходимо внести изменение в класс ResourceManager так, чтобы работа с материалом осуществлялась через ResourceManager. В частности, каждый материал должен храниться в памяти в единственном экземпляре также, как это было сделано для меша и текстуры.
4. Требуется модифицировать класс GraphicObject, добавив в него идентификатор используемого материала.
5. Необходимо модифицировать класс RenderManager так, чтобы он передавал в шейдер все параметры материала и источника света. При передаче направления на источник света из основной программы в шейдер необходимо не забыть перевести его в систему координат наблюдателя (то есть умножить на матрицу наблюдения). Для проверки работоспособности программы рекомендуется использовать шейдер, приведенный в демонстрационном примере.
6. В завершении необходимо модифицировать шейдер расчета освещения по Фонгу, добавив в него учет воздушной перспективы.
7. Ресурсы для демонстрации программы (меши, шейдеры и материалы) необходимо взять из демонстрационного примера.

Отчет к лабораторной работе должен быть сформирован на основе шаблона, представленного на следующей странице. При оформлении отчета необходимо внести свою фамилию, имя, отчество, номер группы, а также заполнить таблицу. При заполнении таблицы необходимо дать краткое описание реализованным модулям, указать количество строк каждого модуля, а также подсчитать общее количество строк во всем проекте.

Архитектура графических систем (6 семестр)

Выполнил: Иванов Иван Иванович

Группа: 00-00

Проверил: Галибин Сергей Владимирович

Задание:

В лабораторной работе №6 необходимо организовать вывод на экран нескольких графических объектов, на каждый из которых накладывается своя текстура и для которых производится расчет освещенности по методу Фонга. Загрузка параметров материала осуществляется из json-файла.

Защита лабораторной работы:

Для защиты лабораторной работы необходимо знать:

1. **[КГ]** Какие компоненты освещенности есть в моделе освещения Фонга.
2. **[КГ]** Что такое точечные и направленные источники света.
3. **[КГ]** В чем заключается различие в преобразовании позиций и направлений.
4. **[CPP]** Приемы разбора json-файла с использование библиотеки RapidJson.

Состав проекта:

Название модуля и его назначение	**Количество строк**	
	***.h**	***.cpp**
main - основная программа		
Shader -		
Camera -		
GraphicObject -		
Mesh -		
ResourceManager -		
Texture -		
RenderManager –		
Material -		
Light -		